Entregables · Alpha Centauri · Rocket Landing

Proyecto final del curso de MLOps · noviembre de 2026 · equipo Alpha Centauri (mjcastano29-tech, Isaías Céspedes, jhrojas02).

Este documento es el índice de la entrega: qué pide cada criterio de la rúbrica, dónde está la evidencia y cómo se comprueba. Todo lo que se cita vive en el repositorio en el commit etiquetado como `v1.1.0`; los PDF de la carpeta `entregables/` son copias de lectura generadas con `python entregables.py` a partir de estos mismos archivos.

Dónde está todo

Qué	Dónde
Repositorio (código, fichas, modelos, datos, MLflow)	https://github.com/mjcastano29-tech/Alpha-Centauri · etiqueta `v1.1.0`
Web pública (simulador, experimentos, catálogo, incidente, monitoreo, viento en vivo, landing)	https://alpha-centauri-five.vercel.app
API pública	https://alpha-centauri-api.onrender.com (`/version` , `/docs`)
Registro de modelos MLflow	https://alpha-centauri-mlflow.onrender.com
Informe final	docs/informe-final.md · PDF `entregables/01-informe-final.pdf`
Guion de la defensa (15 min)	docs/guion-defensa.md · PDF `entregables/02-guion-defensa.pdf`
Experimentos, variables y dónde se guarda cada modelo	EXPERIMENTOS.md (generado por `python experimentos_md.py`)
Fichas de los ambientes vigentes	ambiente-v0.1.md (N1), ambiente-v0.4.md (N2 pasarela), ambiente-v0.5.md (N3 pasarela con viento); retirados: v0.2, v0.3
Pipeline	pipelines/README.md, código pipelines/entrenar_publicar.py, evidencia JSON en pipelines/ejecuciones/, captura evidencia-prefect-flow-run.png
Historial de versiones del ambiente, del agente y de la web	CHANGELOG.md
Diagnóstico frente al enunciado, hito a hito	diagnostico-proyecto-final.md

Los servicios públicos están en el plan gratuito de Render: la primera petición tras 15 minutos sin uso tarda cerca de un minuto. Antes de una demostración, abrir `#/monitoreo` y pulsar **Medir latencia ahora**.

Rúbrica → evidencia

1. Versionado (30 %)

Qué se versiona	Evidencia	Cómo comprobarlo
Ambiente, con contrato y versión semántica	`RocketLanding3DEnv.VERSION` y las clases de `rocket_env/`; `compatible()` compara mayor.menor; fichas `docs/ambiente-v0.*.md`; CHANGELOG.md	`python check_env.py`, `python -m pytest tests/test_ambiente.py`
Modelos (pesos, normalizador)	`resultados/runs/<run_id>/modelo.zip` + `modelo_normalizador.pkl`, huella SHA-256 en el manifest; versión registrada en MLflow (`rocket-landing-ppo`, `rocket-landing-qlearning`)	`python cli.py verificar` recalcula las huellas de las 23 ejecuciones
Datos de entrenamiento y evaluación	`episodios.csv`, `entrenamiento.json`, `vuelos.json.gz`, `evaluacion.json` por ejecución; suites en `suites/`	huellas en el manifest; `python cli.py evaluar --run <id>` reproduce la evaluación
Configuración e hipótesis	`configs/<run_id>.json` (hipótesis, baseline, presupuesto, decisión con fecha); `manifests/<run_id>.json` con el formato de la guía (sección 11: `git_commit`, `config_sha256`, `model_uri`, `model_sha256`, `evaluation_suite_id`, `seed`, `parent_run_id`)	`python manifiestos.py`; `GET /ejecuciones/<id>/manifiesto`
Registro de experimentos	`mlflow.db` y `mlruns/` dentro del repositorio; UI pública en Render	abrir una run desde la ficha del modelo en la web
Catálogo de lo que se sirve	`catalogo.json` v15 con recomendado y anterior por nivel e historial de 15 cambios	`python catalogo.py ver`; `GET /version` dice qué commit y catálogo sirve la API
Entregas etiquetadas	`v0.2.0` (hito 1), `v1.0.0` (cierre del 10 de noviembre), `v1.1.0` (viento en vivo, pantalla completa, paquete de entregables)	`git tag -l`

2. Diseño experimental (25 %)

Qué	Evidencia
Hipótesis, presupuesto y decisión por configuración	`configs/*.json`; resumen en EXPERIMENTOS.md; cadena de hipótesis en la sección 3 del informe
6 experimentos, 23 ejecuciones, rechazos conservados	`resultados/registro.json`; `experimentos.py`; vista `#/experimentos` de la web, con la progresión de complejidad y las variables de cada experimento
Tres semillas de cada finalista y medias con desviación	informe, sección 2 (97,8 ± 1,0 / 96,3 ± 4,4 / 93,7 ± 1,0 %)

Qué	Evidencia
Suite de evaluación fija y test reservado de una sola vez	`suites/` ; `suite_reservada.py` con la semilla fuera del repositorio; resultados en `resultados/suite-reservada/` (97,5 / 99,0 / 97,0 %)
Incidente controlado e intervención	`incidente.py` , 9 medidas en `resultados/incidentes/` ; vista `#/incidente`

3. Pipeline, despliegue y rollback (20 %)

Qué	Evidencia
Pipeline orquestado de 8 etapas con criterio de aceptación fijado antes de entrenar	pipelines/README.md; 21 evidencias JSON en `pipelines/ejecuciones/` ; captura de Prefect
Despliegue continuo	push a `main` → Render (API y MLflow, render.yaml) y Vercel (web); `GET /version` confirma el commit
Rollback demostrado en producción	`python catalogo.py restaurar --nivel N3` ; evidencia `pipelines/ejecuciones/rollback-n3-*.json` con `comprobar_api.py` ; sección 5 del informe
Monitoreo	`GET /monitoreo` y vista `#/monitoreo` : servicio, modelo servido por nivel, decisiones del pipeline, latencia, alertas
Servicio del modelo	`POST /evaluar` (evaluación con el modelo congelado), `POST /sesiones` (misión paso a paso con viento manual)

4. Contrato del ambiente (10 %)

Qué	Evidencia
Ficha del ambiente (observación, acción, recompensa, terminación, versión, mapa)	ambiente-v0.1.md, ambiente-v0.4.md, ambiente-v0.5.md; mapas SVG en `docs/`
`terminated` frente a `truncated`	éxito y fallos terminan; agotar 50 pasos trunca; pruebas en `tests/test_ambiente.py`
Pruebas de contrato	`python -m pytest` (113 pruebas: ambiente, modelos, API)

5. Defensa (15 %)

Qué	Evidencia
Guion de 15 minutos sobre la web pública, con reparto y preguntas probables	guion-defensa.md
Demostraciones en vivo	replay 3D por nivel (`#/`), experimentos (`#/experimentos`), evaluación en vivo desde la ficha, viento en vivo (`#/vivo`), incidente, monitoreo y rollback
Landing de presentación	https://alpha-centauri-five.vercel.app/landing/

Cómo reproducir desde cero

```
git clone https://github.com/mjcastano29-tech/Alpha-Centauri && cd Alpha-Centauri && git checl
python -m venv .venv && .venv/bin/pip install -r requirements.txt
python -m pytest                        # 113 pruebas
python cli.py verificar                 # huellas de las 23 ejecuciones
python cli.py evaluar --run ppo-pasarela-viento-transfer-1m5-desde-s2-v0.5-s0 --viento 15
uvicorn api.main:app --port 8000        # API local; web: cd web && npm ci && npm run dev
python entregables.py                   # regenera los PDF de entregables/
```

Lo que queda en manos del equipo

- Escribir los nombres completos de los tres integrantes en la cabecera del informe (hoy figuran los usuarios de GitHub).
- Decidir el nombre con el que se presenta la web: la aplicación dice "PROJECT ELYNDRA · Alfa Centauri" y la landing "Alpha Centauri".
- Guardar la semilla del test reservado fuera del repositorio (archivo `.suite_reservada`) en la máquina que vaya a repetirlo.
- Ensayar el guion con la web pública despierta.